Name: Sk Arshad Basha

RegNo: 192312336

1. You are given a large dataset of patient health records stored in a CSV file, containing missing values, inconsistent formats, and outliers. Using NumPy and Pandas, design a constraint-based data cleaning pipeline that ensures: (i) no missing values remain, (ii) all numerical values fall within medically valid ranges, and (iii) categorical fields are standardized. Explain how you will use indexing, filtering, and aggregation functions to enforce these constraints while preserving maximum data integrity.

Question 1: Constraint-Based Data Cleaning Pipeline Using NumPy and Pandas

Introduction

A large patient health-record dataset may contain missing values, inconsistent formats, duplicate records, and outliers. A constraint-based data cleaning pipeline applies predefined rules to make the data accurate, consistent, and suitable for analysis.

The main constraints are:

1. No missing values remain.
2. Numerical values are within medically valid ranges.
3. Categorical fields are standardized.
4. Duplicate records are identified.
5. Maximum valid information is preserved.

Step 1: Load the Dataset

```
import pandas as pd

import numpy as np

df = pd.read_csv("patients.csv")

print(df.head())

print(df.info())

print(df.isnull().sum())
```

head() displays sample records, info() displays data types, and isnull().sum() identifies missing values.

Step 2: Handle Missing Values

For numerical columns, median values can be used:

```
num_cols = ["Age", "Temperature", "Heart_Rate"]

for col in num_cols:
    df[col] = df[col].fillna(df[col].median())
```

For categorical columns, the mode can be used:

```
cat_cols = ["Gender", "Blood_Group"]


for col in cat_cols:
    df[col] = df[col].fillna(df[col].mode()[0])
```

This preserves valid patient records instead of deleting them unnecessarily.

Step 3: Standardize Categorical Data

Inconsistent values such as Male, male, and MALE should be standardized.

```
df["Gender"] = (
    df["Gender"]
    .astype(str)
    .str.strip()
    .str.lower()
)
df["Gender"] = df["Gender"].replace({
    "m": "male",
    "f": "female"
})
```

Blood groups can be standardized as:

```
df["Blood_Group"] = (
```

```
    df["Blood_Group"]
    .astype(str)
    .str.strip()
    .str.upper()
)
```

Conclusion

The constraint-based pipeline uses indexing, filtering, replacement, grouping, and aggregation to clean patient data. Missing values are handled appropriately, invalid numerical values are removed or corrected, and categorical fields are standardized. This approach maintains maximum data integrity while satisfying all required constraints.

2. A system with limited memory must process a massive numerical dataset using NumPy arrays. You are required to perform mathematical operations, random sampling, and statistical analysis without exceeding memory limits. Propose a solution that uses efficient array creation, slicing, and in-place operations. Discuss how indexing and aggregation can be optimized to minimize memory usage, and explain debugging strategies if memory overflow or performance bottlenecks occur.

Introduction

A massive numerical dataset may not fit completely into available memory. NumPy provides efficient arrays, slicing, vectorization, random sampling, and in-place operations that reduce memory consumption.

The main objectives are:

- Minimize memory usage.
- Avoid unnecessary copies.
- Perform calculations efficiently.
- Optimize indexing and aggregation.
- Identify memory and performance problems.

Step 1: Efficient Array Creation

```
import numpy as np
```

```
data = np.array(
    [1, 2, 3, 4, 5],
    dtype=np.float32
)
```

Using float32 requires less memory than float64.

For a large array:

```
data = np.zeros(
    1000000,
    dtype=np.float32
)
```

Step 2: Use Slicing

Only the required part of an array should be processed:

```
part = data[100000:200000]
```

Regular sampling can be performed efficiently:

```
sample = data[::10]
```

This selects every tenth element.

Step 3: Use In-Place Operations

Instead of:

```
data = data * 2
```

use:

```
data *= 2
```

Other in-place operations include:

```
data += 10
data -= 5
data /= 2
```

These reduce unnecessary memory allocation.

Step 4: Random Sampling

```
rng = np.random.default_rng(42)


sample = rng.choice(
    data,
    size=1000,
    replace=False
)
```

Only the required sample is stored.

Conclusion

Massive numerical datasets can be efficiently processed using appropriate data types, slicing, in-place operations, random sampling, chunk processing, and aggregation. Memory usage can be monitored with nbytes, while execution time can be measured to identify performance bottlenecks.

3. Two large Pandas DataFrames containing customer transaction and demographic data must be combined. However, constraints include: (i) no duplication of customer IDs, (ii) consistent indexing after merging, and (iii) preservation of all valid records. Describe how you will perform the combining operation, validate constraints using filtering and grouping, and ensure correctness through sorting and indexing techniques. Include steps to debug mismatched or missing entries.

Introduction

Customer transaction and demographic information may be stored in separate DataFrames. These datasets must be combined while maintaining customer-ID consistency and preserving valid records.

The main constraints are:

1. No incorrect duplication of customer IDs.
2. Consistent indexing.
3. Preservation of valid records.
4. Detection of mismatched or missing entries.

Step 1: Load DataFrames

```
import pandas as pd

transactions = pd.read_csv("transactions.csv")

demographic = pd.read_csv("demographic.csv")
```

Step 2: Check Customer IDs

```
print(transactions["Customer_ID"].is_unique)

print(demographic["Customer_ID"].is_unique)
```

This identifies whether the customer IDs are unique.

Step 3: Detect Duplicate IDs

```
duplicates = demographic[
    demographic["Customer_ID"].duplicated(
        keep=False
    )
]

print(duplicates)
```

Duplicates can be removed if only one demographic record per customer is valid:

```
demographic = demographic.drop_duplicates(
    subset="Customer_ID",
    keep="first"
)
```

Conclusion

Pandas merge() can combine large DataFrames accurately when customer IDs are standardized and validated. Duplicate detection, filtering, grouping, sorting, and index resetting help maintain data integrity. Debugging mismatched IDs ensures that valid customer records are not lost.

4. You are tasked with computing statistical measures (mean, median, variance) on a dataset using NumPy and Pandas, but with constraints on both

computational efficiency and numerical accuracy. Explain how you will implement aggregation and statistical functions while avoiding precision loss and redundant computations. Discuss how filtering and indexing can be used to exclude irrelevant data points, and how you would debug incorrect statistical outputs.

Introduction

Mean, median, and variance are important statistical measures for understanding data. NumPy and Pandas provide efficient aggregation functions for calculating these measures.

The main requirements are:

- Accurate calculations.
- Efficient aggregation.
- Proper filtering.
- Correct handling of missing values.
- Avoidance of redundant computations.

Step 1: Create Dataset

```
import numpy as np
import pandas as pd
data = np.array(
    [10, 20, 30, 40, 50],
    dtype=np.float64
)

df = pd.DataFrame({
    "Value": data
})
```

Step 2: Calculate Mean

The mean is:

[
Mean = \frac{\sum x_i}{n}
]

Using NumPy:

mean = np.mean(data)

Using Pandas:

mean = df["Value"].mean()

Step 3: Calculate Median

The median is the middle value after sorting.

median = np.median(data)

Using Pandas:

median = df["Value"].median()

Median is generally less affected by extreme values than the mean.

Conclusion

NumPy and Pandas provide efficient functions for calculating **mean, median, and variance**. Indexing and filtering allow irrelevant data to be removed before analysis. Appropriate numerical data types improve precision, while aggregation avoids unnecessary calculations. Debugging using count(), describe(), min(), and max() helps verify statistical accuracy.

5. Design a complete data analysis pipeline that reads multiple data files, processes them using Pandas Data Frames, and generates visualizations. Constraints include: (i) efficient file I/O operations, (ii) correct grouping and sorting of data, (iii) filtering based on user-defined conditions, and (iv) meaningful plotting outputs. Explain how each stage (loading, processing, aggregation, and plotting) will be implemented and optimized, along with debugging steps for handling corrupted files or inconsistent data formats.

Introduction

A complete data analysis pipeline processes raw data files and converts them into meaningful information. The major stages are:

Data Files

↓

Loading

↓

Validation

↓

Cleaning

↓

Filtering

↓

Grouping

↓

Aggregation

↓

Sorting

↓

Visualization

↓

Final Analysis

Step 1: Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import glob
```

Step 2: Find Multiple Files

```
files = glob.glob("data/*.csv")
```

This identifies all CSV files in the selected folder.

Step 3: Load Files

```
frames = []
for file in files:
    try:
        df = pd.read_csv(file)
        frames.append(df)
    except Exception as e:
        print("Error:", file, e)
```

Error handling prevents one corrupted file from stopping the complete analysis.

Step 4: Combine Data

```
data = pd.concat(
    frames,
    ignore_index=True
)
```

This combines all DataFrames and creates a continuous index.

Step 5: Inspect Data

```
print(data.head())
print(data.shape)
print(data.info())
print(data.describe())
```

These functions help identify the structure, data types, missing values, and statistical properties.

Step 6: Standardize Column Names

```
data.columns = (
    data.columns
    .str.strip()
    .str.lower()
```

```
    .str.replace(" ", "_")
)
```

This creates consistent column names.

Step 7: Handle Missing Values

Numerical columns:

```
data["sales"] = data["sales"].fillna(
    data["sales"].median()
)
```

Categorical columns:

```
data["category"] = data["category"].fillna(
    data["category"].mode()[0]
)
```

Step 8: Apply User-Defined Filtering

For example, selecting sales greater than 10,000:

```
threshold = 10000
filtered = data.loc[
    data["sales"] > threshold
].copy()
```

Step 9: Grouping

```
grouped = (
    filtered
    .groupby("category")["sales"]
    .sum()
)
```

This calculates total sales for each category.

Line Chart

```
data.plot(
    x="date",
    y="sales",
    kind="line"
)
plt.title("Sales Trend")
plt.show()
```

Step 13: Efficient File I/O

For very large files, use chunks:

```
for chunk in pd.read_csv(
    "large_file.csv",
    chunksize=50000
):
    # Process each chunk
    pass
```

Only a limited amount of data is loaded at a time.

Specific columns can also be selected:

```
df = pd.read_csv(
    "data.csv",
    usecols=["date", "category", "sales"]
)
```

This reduces memory consumption.

Step 14: Debugging Corrupted Files

```
try:
    df = pd.read_csv(file)
```

```
except pd.errors.ParserError:
    print("Parsing error:", file)

except UnicodeDecodeError:
    print("Encoding error:", file)

except Exception as e:
    print("Other error:", e)
```

Step 15: Debugging Inconsistent Data

Check column names:

```
print(df.columns)
```

Check data types:

```
print(df.dtypes)
```

Check missing values:

```
print(df.isnull().sum())
```

Check duplicate records:

```
print(df.duplicated().sum())
```

Check categorical values:

```
print(df["category"].unique())
```

Conclusion

A complete data-analysis pipeline consists of loading, validation, cleaning, filtering, grouping, aggregation, sorting, and visualization. Efficient file I/O, chunk processing, appropriate data types, and column selection reduce memory usage. Filtering and grouping produce meaningful results, while error handling and validation help identify corrupted files and inconsistent data. This creates a reliable and efficient data-analysis system.